UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

ECE408 - CS483 - CSE408 Spring 2026
Applied Parallel Programming

Lecture 3: CUDA Parallel Execution Model: Multidimensional Grids & Data

Volodymyr Kindratenko

1/27/2026

Course Reminders

- Lab 0 was due last Friday at 8pm US Central time
  - This lab does not contribute to your overall grade, but you still must do it
- Lab 1 is due this Friday
  - Requires Lab 0 to be completed first
  - Easy lab, all code was given, however do not postpone doing it until last minute
  - Also not included in the final grade
- Post your questions on Campuswire, the course staff only replies to questions posted on Campuswire
  - But before posting a question, make sure you try to find the answer yourself first, <u>including asking the course chatbot</u>
- Check office hours schedule, we have in-person and on-line

About Delta

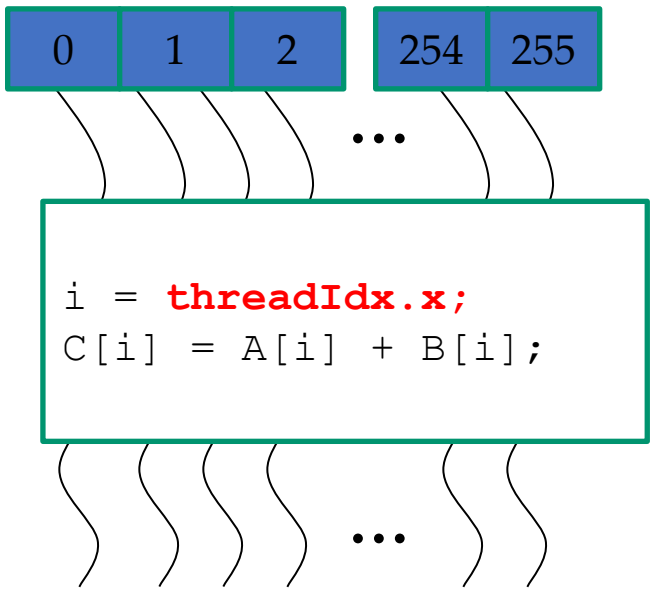
- Your Delta system quota for the entire semester is **10 GPU hours**
  - This is enough for all the assignments, unless you do something outside the scope of what is required and expected.
  - We will monitor GPU usage. If a student exceeds 2 GPU hours per lab (which should allow for over 100 runs) or exceeds 10 GPU hours in total, we will reach out to discuss their usage.
  - To check your usage, run `jobcharge -y 2026 --detail`
- General guidelines:
  - **Authorized Use Only:** Using this course's Delta cluster credits for any purpose unrelated to the course is strictly prohibited. If such misuse is detected, we reserve the right to revoke your Delta access.
  - **Proper Debugging Practices:** Interactive sessions, Jupyter notebooks, and OnDemand sessions are not prohibited and may provide a better debugging experience in some cases. However, students should be aware that these sessions consume GPU credits quickly. Please use them judiciously and prefer developing code on the login node.
  - **Conserving Resources:** Please be mindful of the Delta credits you use. If we run out before the semester ends, we may have to transition to the other clusters or Google Colab, which requires additional setup and adaptation. We want to avoid this scenario.

Today's Objectives

- To learn more about the multi-dimensional logical organization of CUDA threads
- To learn to use control structures, such as loops in a kernel

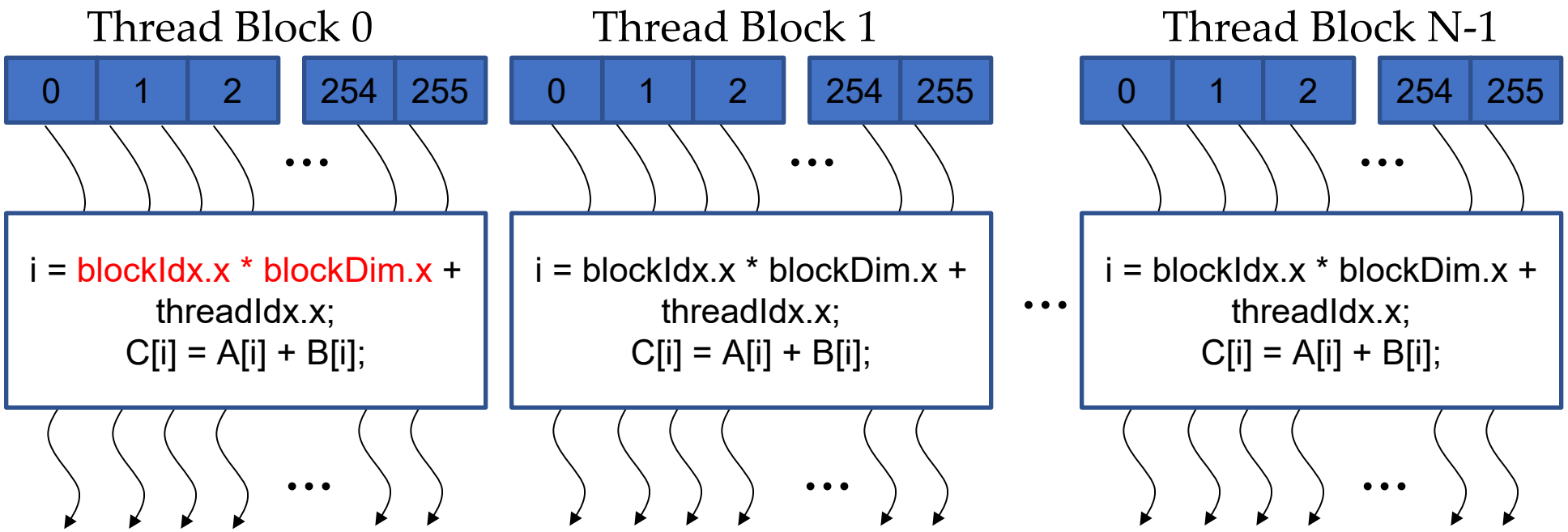
Review: Arrays of Parallel Threads

- A CUDA kernel is executed as a grid (array) of threads
  - All threads in a grid run the same kernel code
  - Single Program Multiple Data (SPMD model)
  - Each thread has a **unique index** that it uses to compute memory addresses and make control decisions



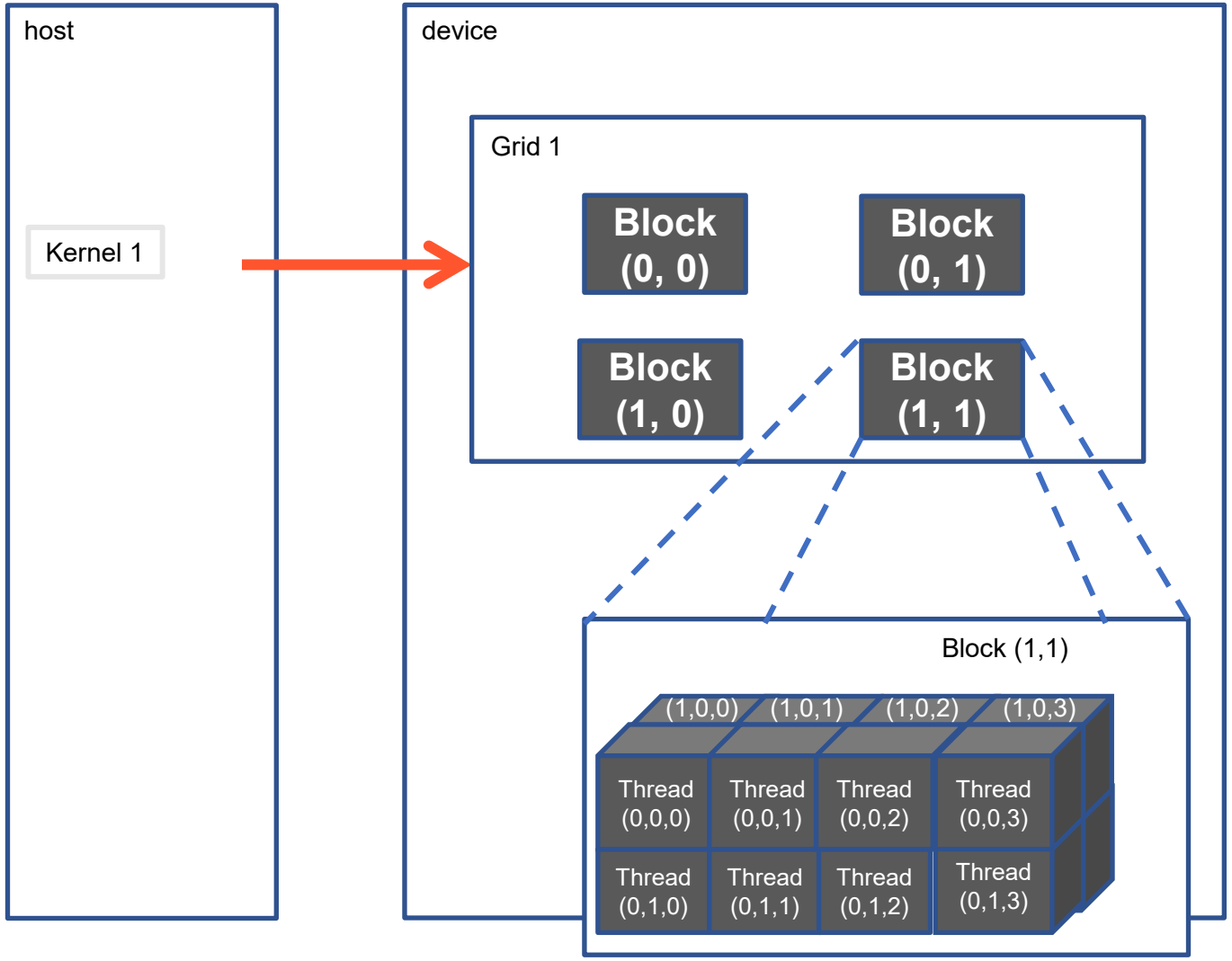
Review: Thread Blocks: Scalable Cooperation

- Threads within a block cooperate via **shared memory, atomic operations** and **barrier synchronization** (to be covered later)
- Threads in different blocks cooperate less.



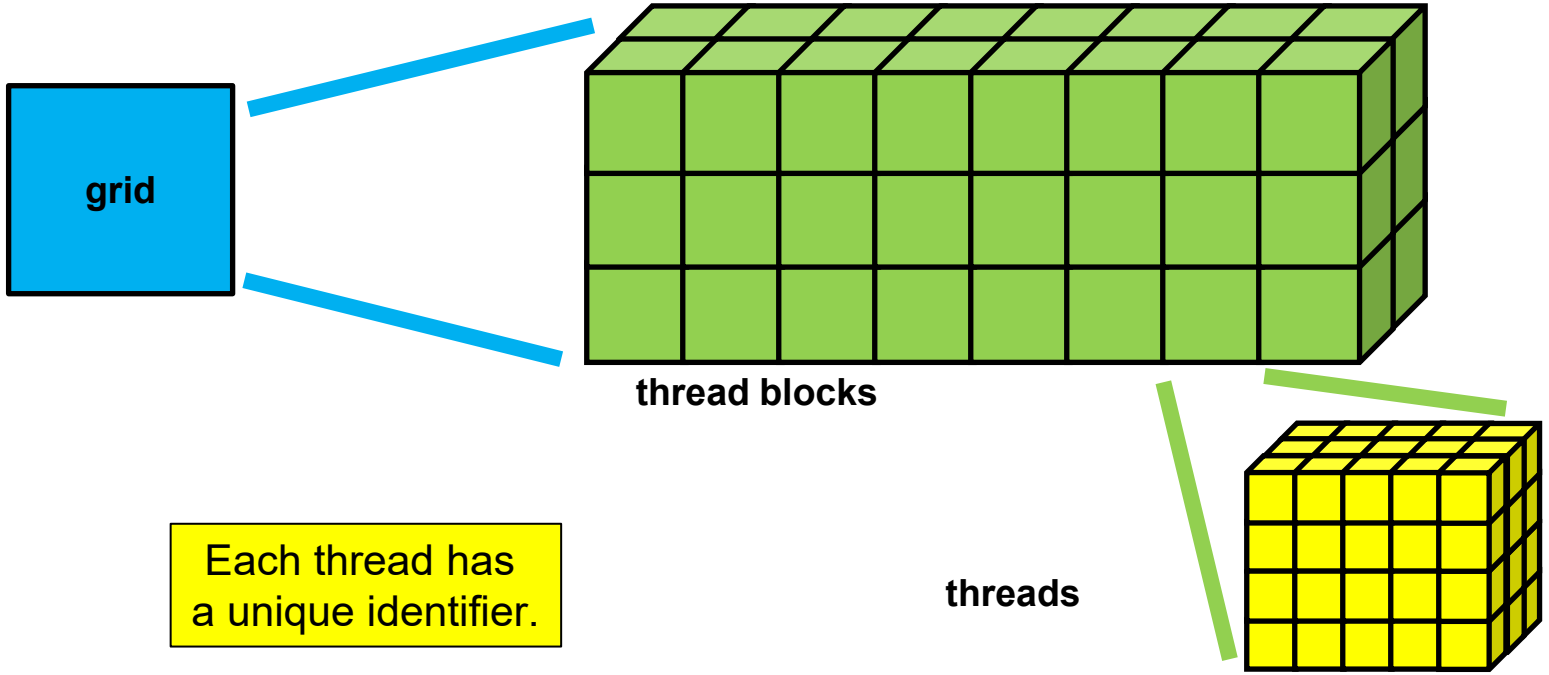
CUDA Thread Grids are Multi-Dimensional

- CUDA supports multidimensional grids (up to 3D)
  - simplifies memory addressing
  - when processing multidimensional data



Logical Execution Model for CUDA

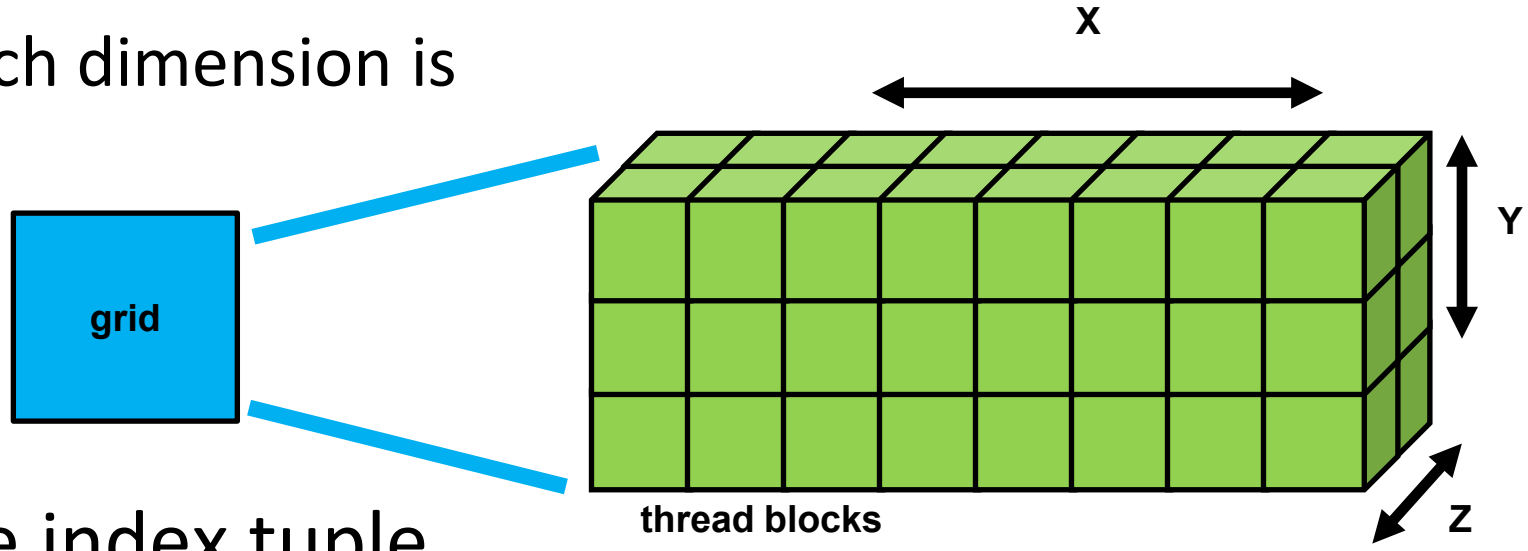
- Each CUDA kernel is executed by a **grid**,
  - a 3D array of **thread blocks**, which are
  - 3D arrays of **threads**.
- Each thread
  - executes the **same program**
  - on **distinct data inputs**,
  - a single-program, multiple-data (**SPMD**) model



Each thread has a unique identifier.

gridDim and blockIdx

- gridDim gives number of blocks
  - Number of blocks in each dimension is
    - gridDim.x … **8**
    - gridDim.y … **3**
    - gridDim.z … **2**

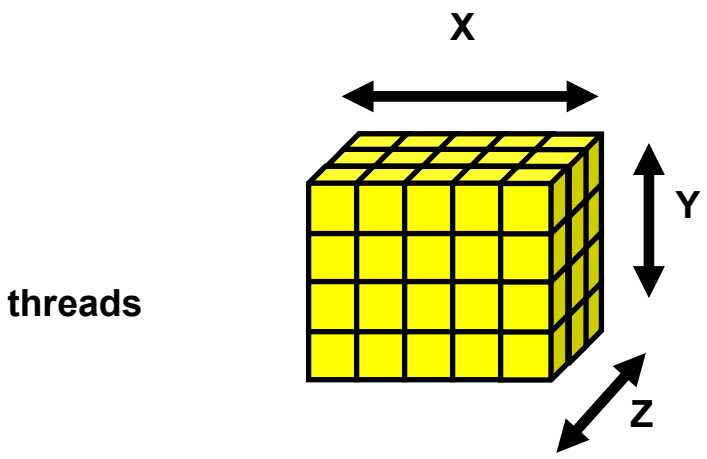


- Each block has a unique index tuple
  - blockIdx.x (from 0 to (gridDim.x – 1))
  - blockIdx.y (from 0 to (gridDim.y – 1))
  - blockIdx.z (from 0 to (gridDim.z – 1))

For 2D (and 1D grids), simply use grid dimension 1 for Z (and Y).

blockDim and threadIdx

- blockDim gives number of threads per block
  - Number of blocks in each dimension is
    - blockDim.x … **5**
    - blockDim.y … **4**
    - blockDim.z … **3**

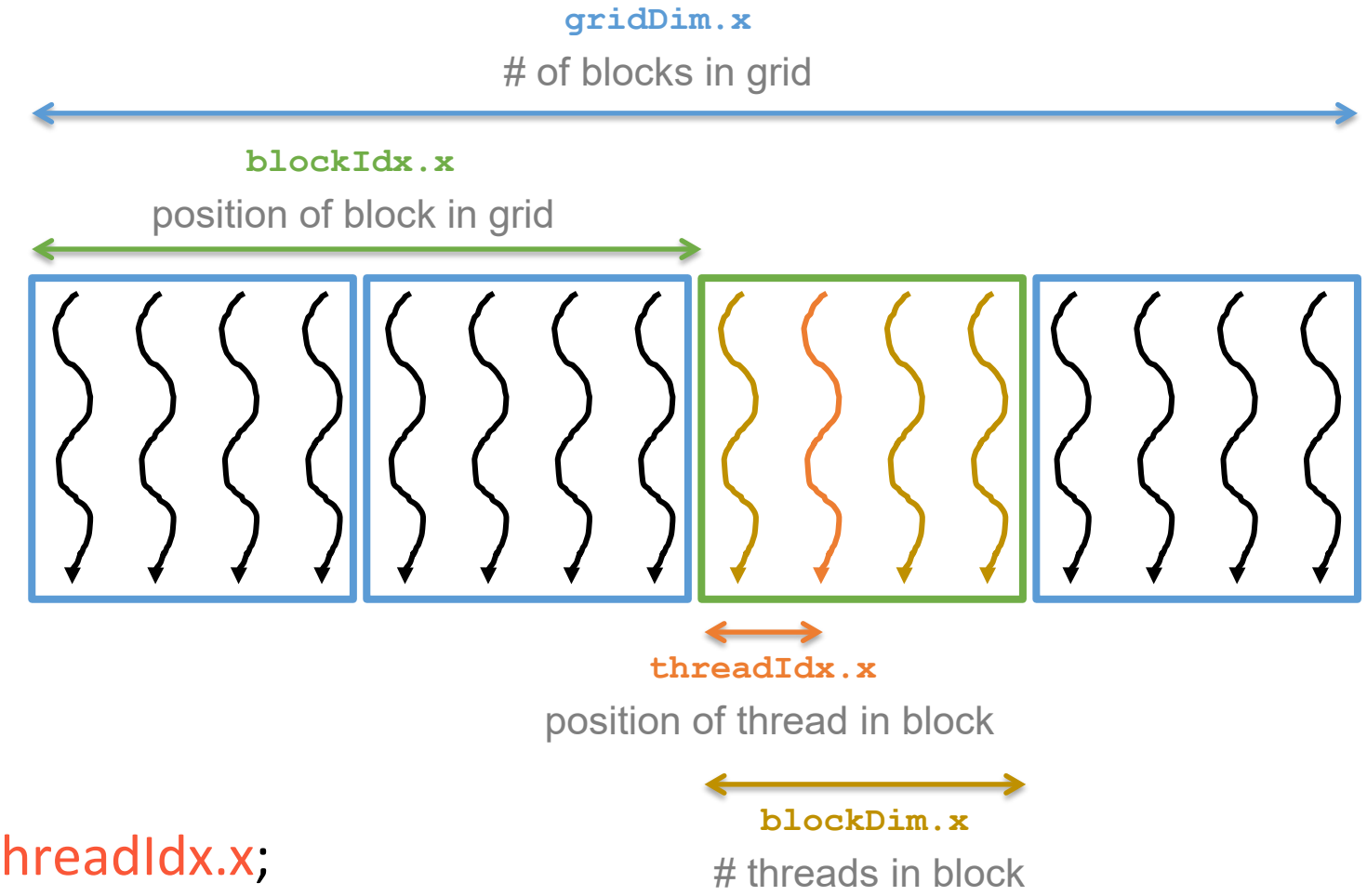


- Each thread has a unique index tuple
  - threadIdx.x (from 0 to (blockDim.x – 1))
  - threadIdx.y (from 0 to (blockDim.y – 1))
  - threadIdx.z (from 0 to (blockDim.z – 1))

For 2D (and 1D blocks), simply use block dimension 1 for Z (and Y).

One Dimensional Indexing

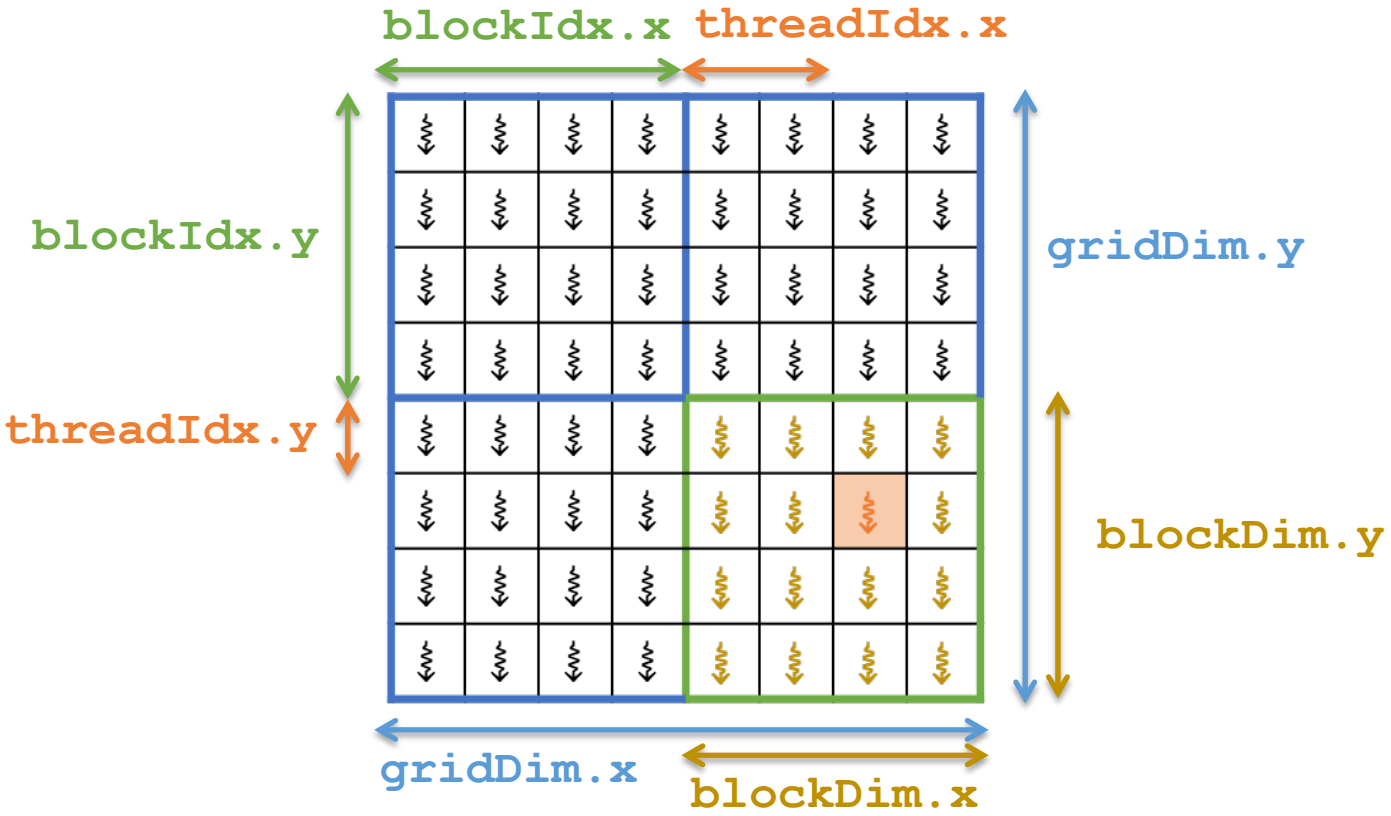
- Thread block and thread organization
  - simplifies memory addressing
  - when processing multidimensional data



- Defining a working set for a thread
  - i = blockIdx.x * blockDim.x + threadIdx.x;

Multidimensional Indexing

- Thread block and thread organization
  - simplifies memory addressing
  - when processing multidimensional data

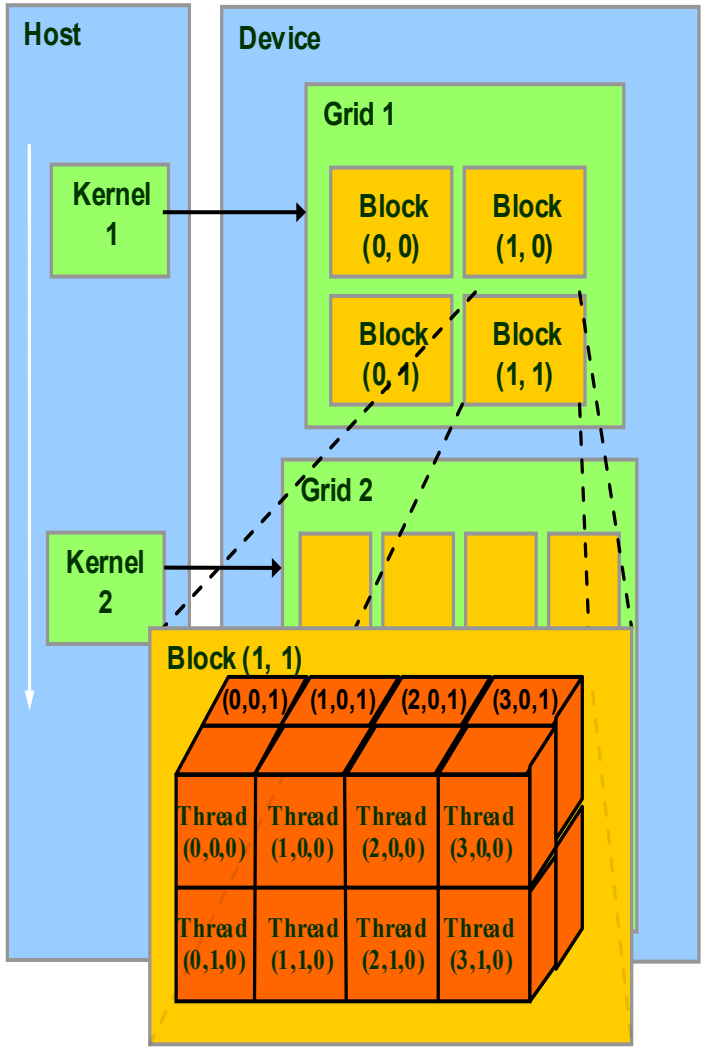


- Defining a working set for a thread
  - row = blockIdx.y * blockDim.y + threadIdx.y;
  - col = blockIdx.x * blockDim.x + threadIdx.x;

Each Kernel Launches Its Own Grid

- Multiple kernels in an application can have their own grid and block dimensions independent of each other

- This is important when working with different types of problems
  - Image processing
  - Vectors, matrices, tensors
  - Solving PDEs on volumes
  - …



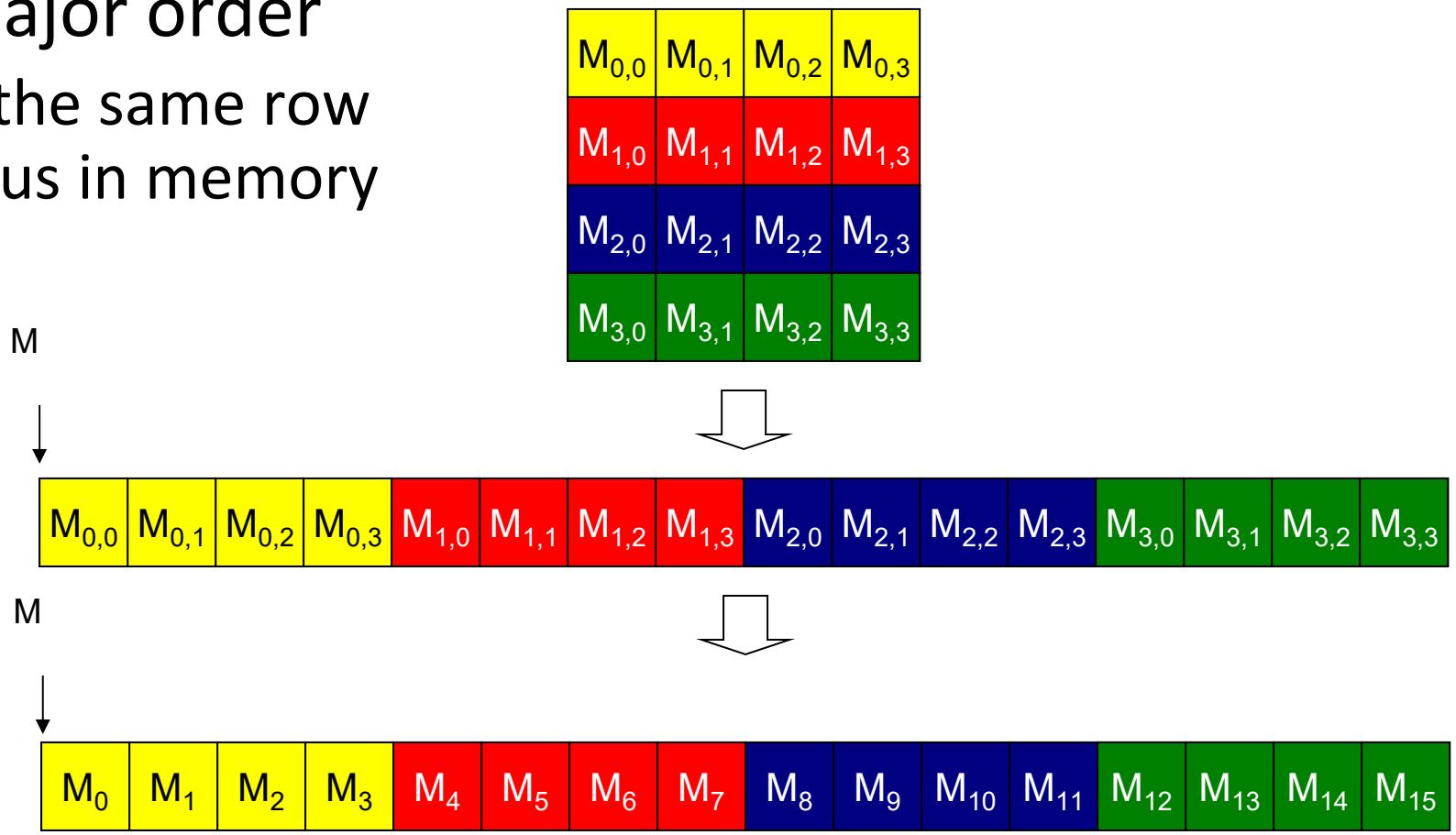
Configuring Multidimensional Grids

- Use built-in `dim3` type
  - The rest is the same

```
dim3 numThreadsPerBlock(32, 32);

dim3 numBlocks(
   (width + numThreadsPerBlock.x - 1) / numThreadsPerBlock.x,
   (height + numThreadsPerBlock.y - 1) / numThreadsPerBlock.y );

kernel <<< numBlocks, numThreadsPerBlock >>> (…kernel args…);
```

Layout of Multidimensional Data

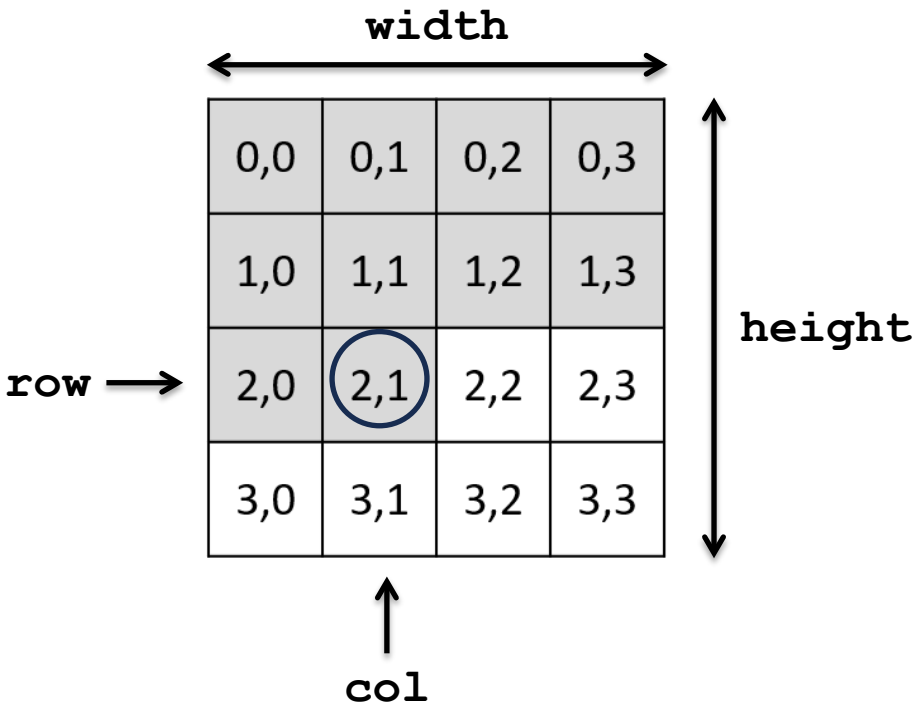
- Convention is C is to store data in row major order
  - Elements in the same row are contiguous in memory



Accessing Multidimensional Data

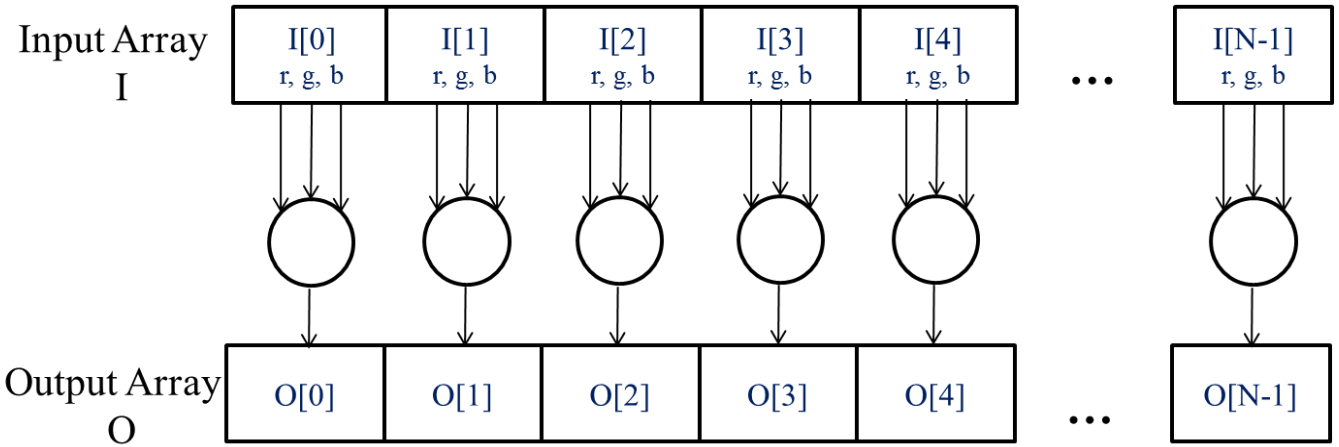
- index = row * width + col

- For example,
  - $M_{2,1}$ → Row*Width+Col = 2*4+1 = 9

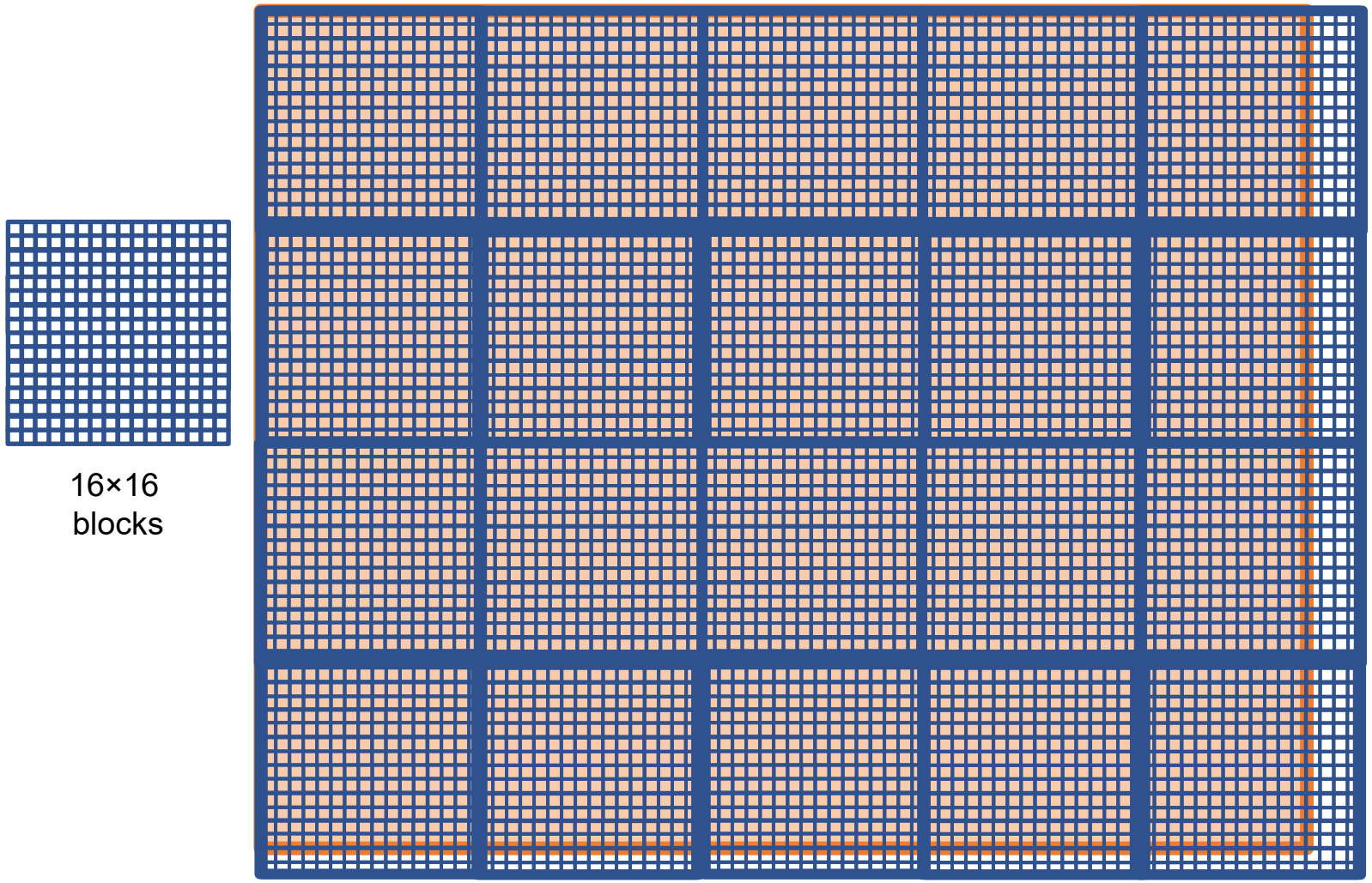


Example: RGB to Gray-Scale Image Conversion

- Parallelization approach: assign one thread to convert each pixel in the image



Processing a Picture with a 2D Grid

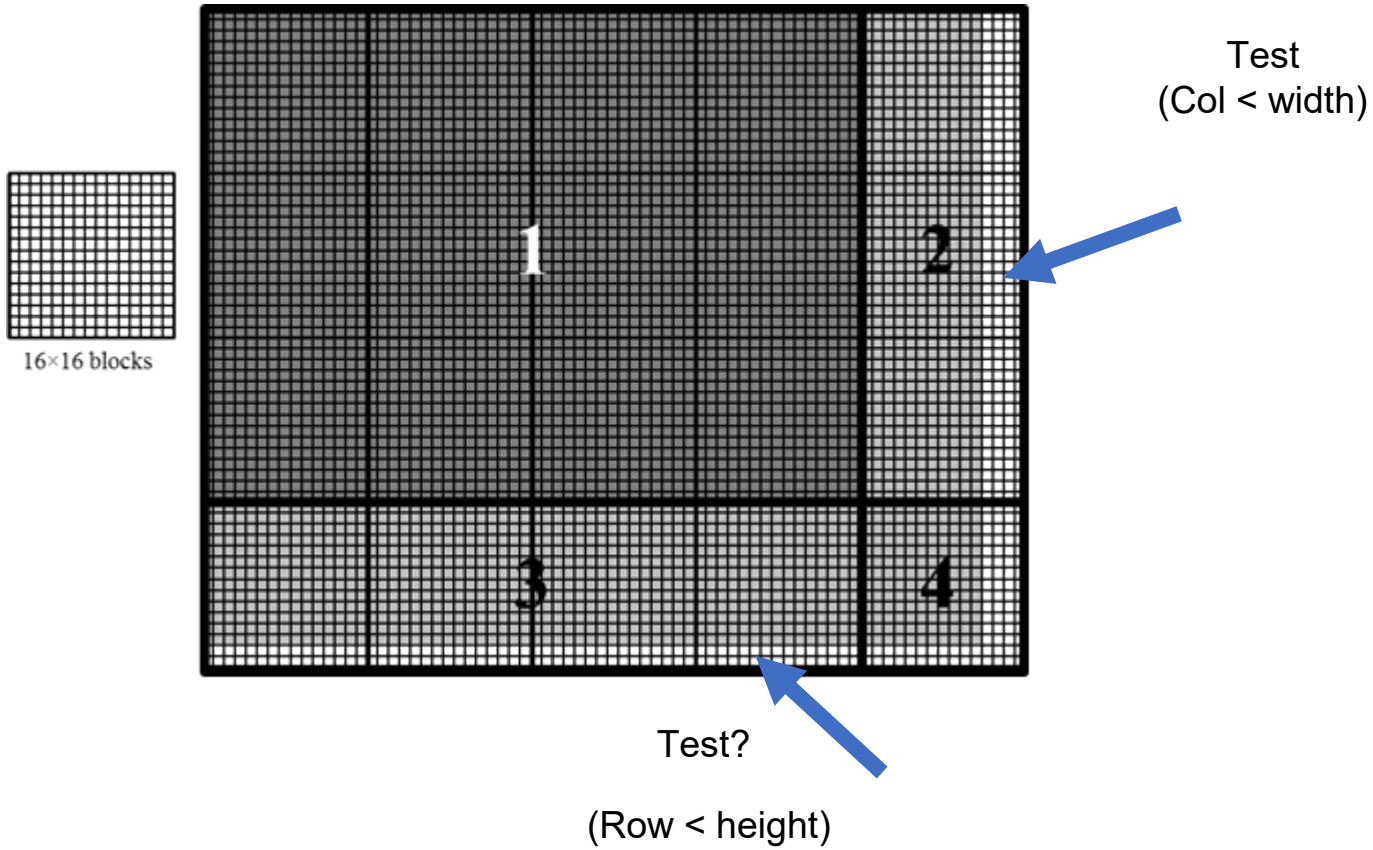


RGB to Gray-Scale Kernel Implementation

```
__global__
void rgb2gray_kernel(unsigned char* red, unsigned char* green, unsigned char* blue,
                     unsigned char* gray, unsigned int width, unsigned int height)
{
    unsigned int row = blockIdx.y*blockDim.y + threadIdx.y;
    unsigned int col = blockIdx.x*blockDim.x + threadIdx.x;

    // Convert the pixel
    gray[row*width + col] = red[row*width + col]*3/10
            + green[row*width + col]*6/10 + blue[row*width + col]*1/10;
}
```

Example: Covering a 76×62 picture with 16×16 blocks

- `int Col = threadIdx.x + blockIdx.x * blockDim.x;`
- `int Row = threadIdx.y + blockIdx.y * blockDim.y;`



Complete RGB to Gray-Scale Kernel Implementation

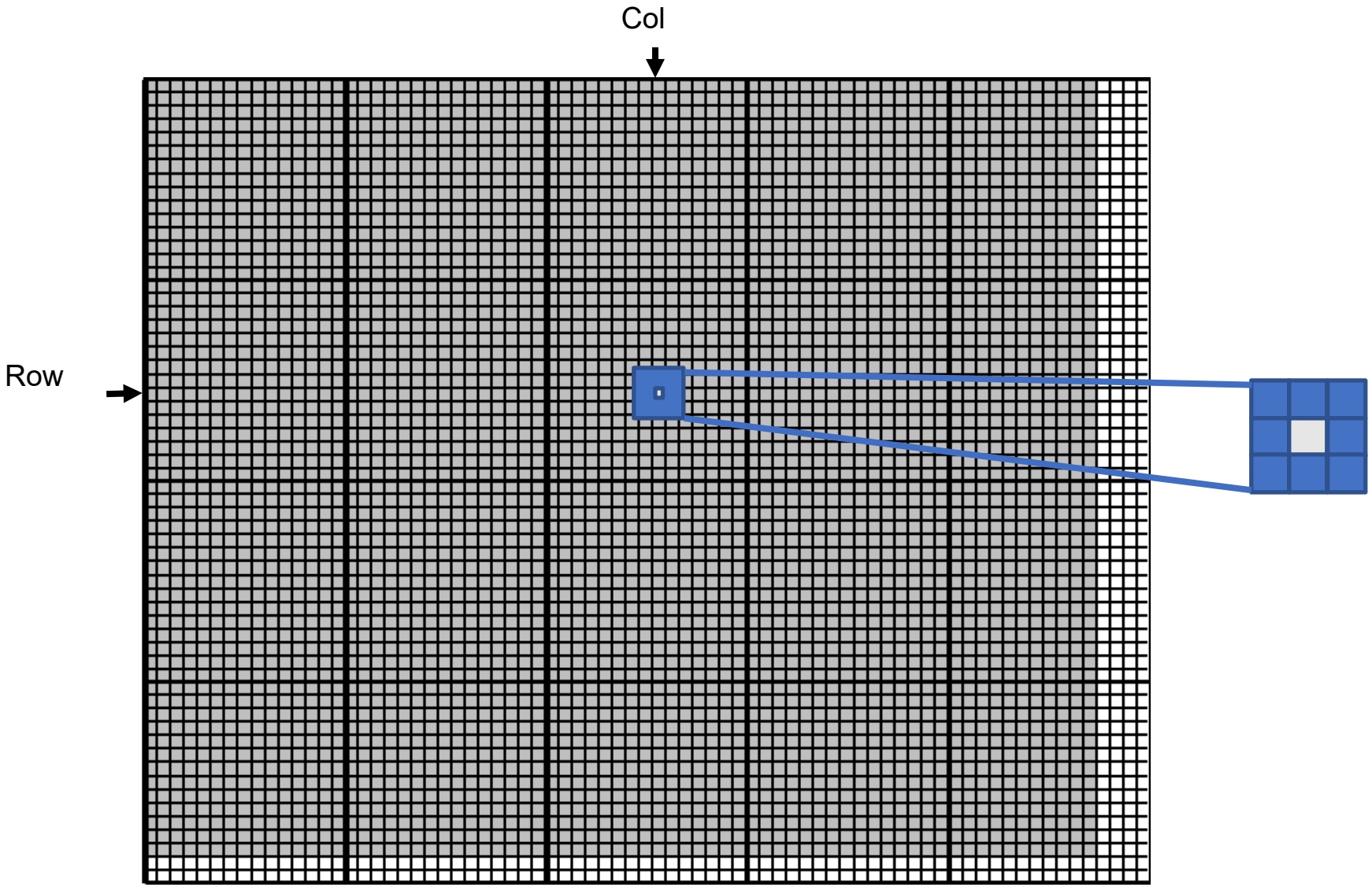
```
__global__
void rgb2gray_kernel(unsigned char* red, unsigned char* green, unsigned char* blue,
                     unsigned char* gray, unsigned int width, unsigned int height)
{
    unsigned int row = blockIdx.y*blockDim.y + threadIdx.y;
    unsigned int col = blockIdx.x*blockDim.x + threadIdx.x;

    // Convert the pixel
    if (row < height && col < width) {
        gray[row*width + col] = red[row*width + col]*3/10
                + green[row*width + col]*6/10 + blue[row*width + col]/10;
    }
}
```

Example: Blur

- Output pixel is the average of the corresponding input pixel and the pixels around it

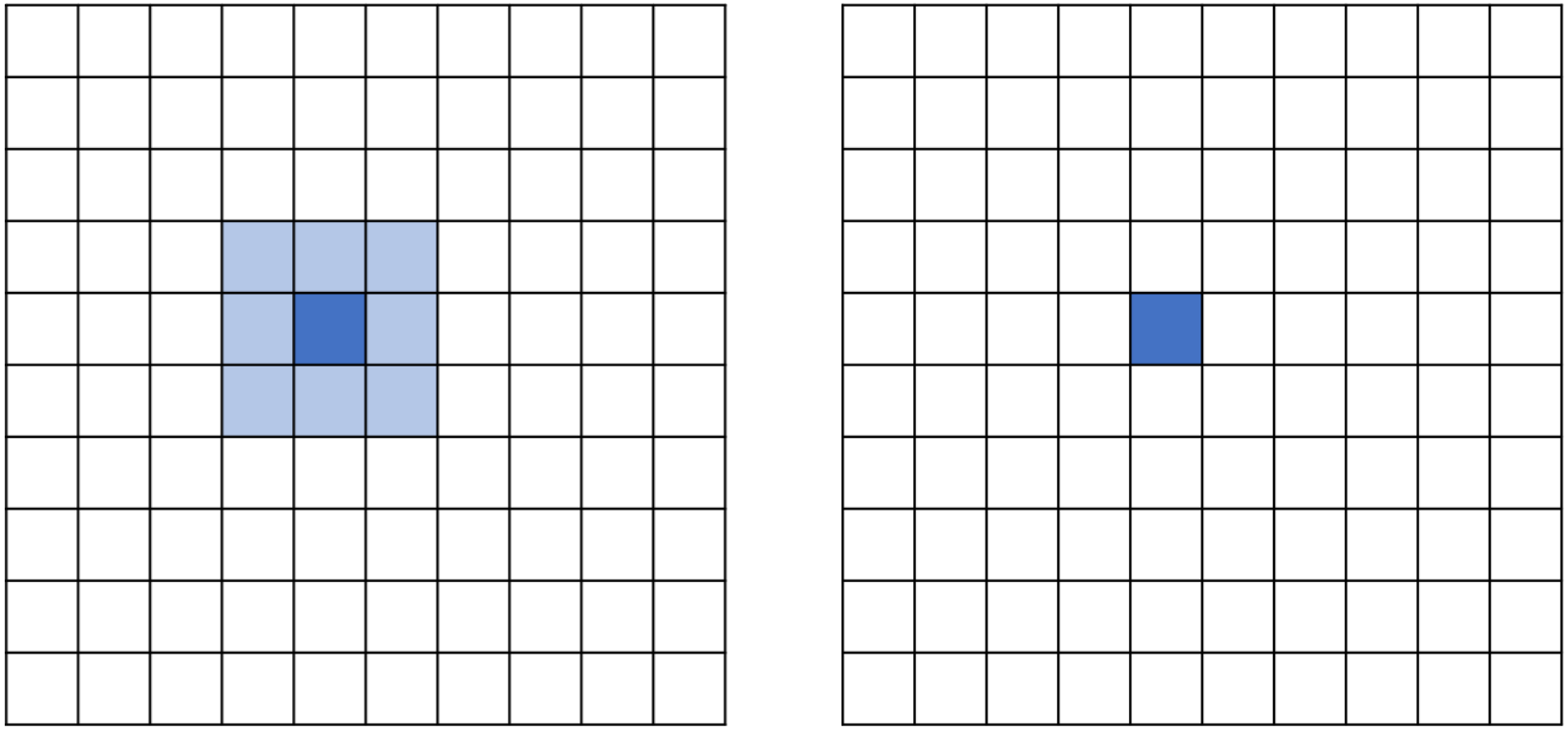
Blur Algorithm

- BLUR_SIZE = 1



Parallelization Strategy

- Parallelization approach: assign one thread to each output pixel, and have it read multiple input pixels



Input Image

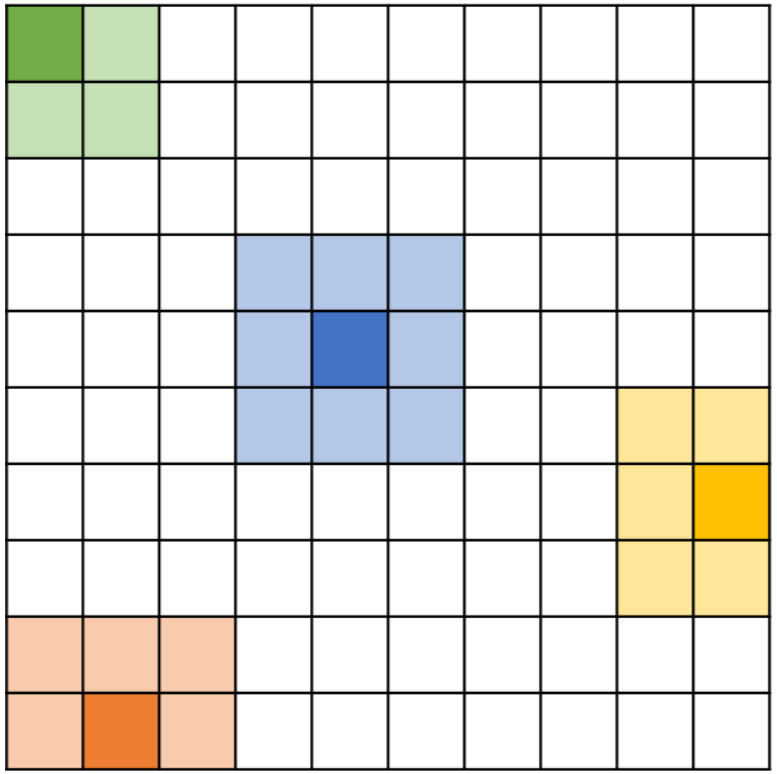
Output Blurred Image

Blur Kernel Implementation (for grayscale image)

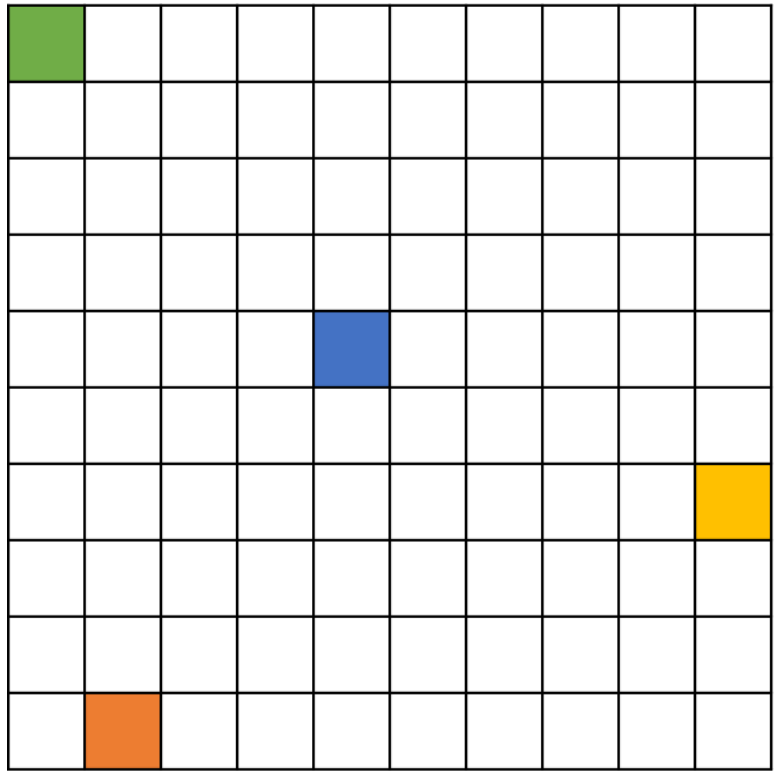
```
__global__ void blur_kernel(unsigned char* image, unsigned char* blurred,
                            unsigned int width, unsigned int height)
{
    int outRow = blockIdx.y*blockDim.y + threadIdx.y;
    int outCol = blockIdx.x*blockDim.x + threadIdx.x;

    if (outRow < height && outCol < width)
    {
        unsigned int average = 0;
        for(int inRow = outRow - BLUR_SIZE; inRow < outRow + BLUR_SIZE + 1; ++inRow) {
            for(int inCol = outCol - BLUR_SIZE; inCol < outCol + BLUR_SIZE + 1; ++inCol) {
                average += image[inRow*width + inCol];
            }
        }

        blurred[outRow*width + outCol] =
                      (unsigned char)(average/((2*BLUR_SIZE + 1)*(2*BLUR_SIZE + 1)));
    }
}
```

Boundary Conditions

- Even if output pixel is in bounds, might be accessing out of bounds input pixel



Input Image



Output Blurred Image

Complete Blur Kernel Implementation (for grayscale image)

```
__global__ void blur_kernel(unsigned char* image, unsigned char* blurred,
                            unsigned int width, unsigned int height)
{
    int outRow = blockIdx.y*blockDim.y + threadIdx.y;
    int outCol = blockIdx.x*blockDim.x + threadIdx.x;

    if (outRow < height && outCol < width)
    {
        unsigned int average = 0;
        for(int inRow = outRow - BLUR_SIZE; inRow < outRow + BLUR_SIZE + 1; ++inRow) {
            for(int inCol = outCol - BLUR_SIZE; inCol < outCol + BLUR_SIZE + 1; ++inCol) {
                if (inRow >= 0 && inRow < height && inCol >= 0 && inCol < width) {
                    average += image[inRow*width + inCol];
                }
            }
        }
        blurred[outRow*width + outCol] =
                      (unsigned char)(average/((2*BLUR_SIZE + 1)*(2*BLUR_SIZE + 1)));
    }

}
```

Rule of thumb: every memory access must have a corresponding guard that compares its indexes to the array dimensions

Complete Blur Kernel Implementation (for grayscale image)

```
__global__ void blur_kernel(unsigned char* image, unsigned char* blurred,
                            unsigned int width, unsigned int height)
{
    int outRow = blockIdx.y*blockDim.y + threadIdx.y;
    int outCol = blockIdx.x*blockDim.x + threadIdx.x;

    if (outRow < height && outCol < width)
    {
        unsigned int average = 0;
        unsigned int count = 0;
        for(int inRow = outRow - BLUR_SIZE; inRow < outRow + BLUR_SIZE + 1; ++inRow) {
            for(int inCol = outCol - BLUR_SIZE; inCol < outCol + BLUR_SIZE + 1; ++inCol) {
                if (inRow >= 0 && inRow < height && inCol >= 0 && inCol < width) {
                    average += image[inRow*width + inCol];
                    count++;
                }
            }
        }
        blurred[outRow*width + outCol] = (unsigned char)(average/count);
    }

}
```

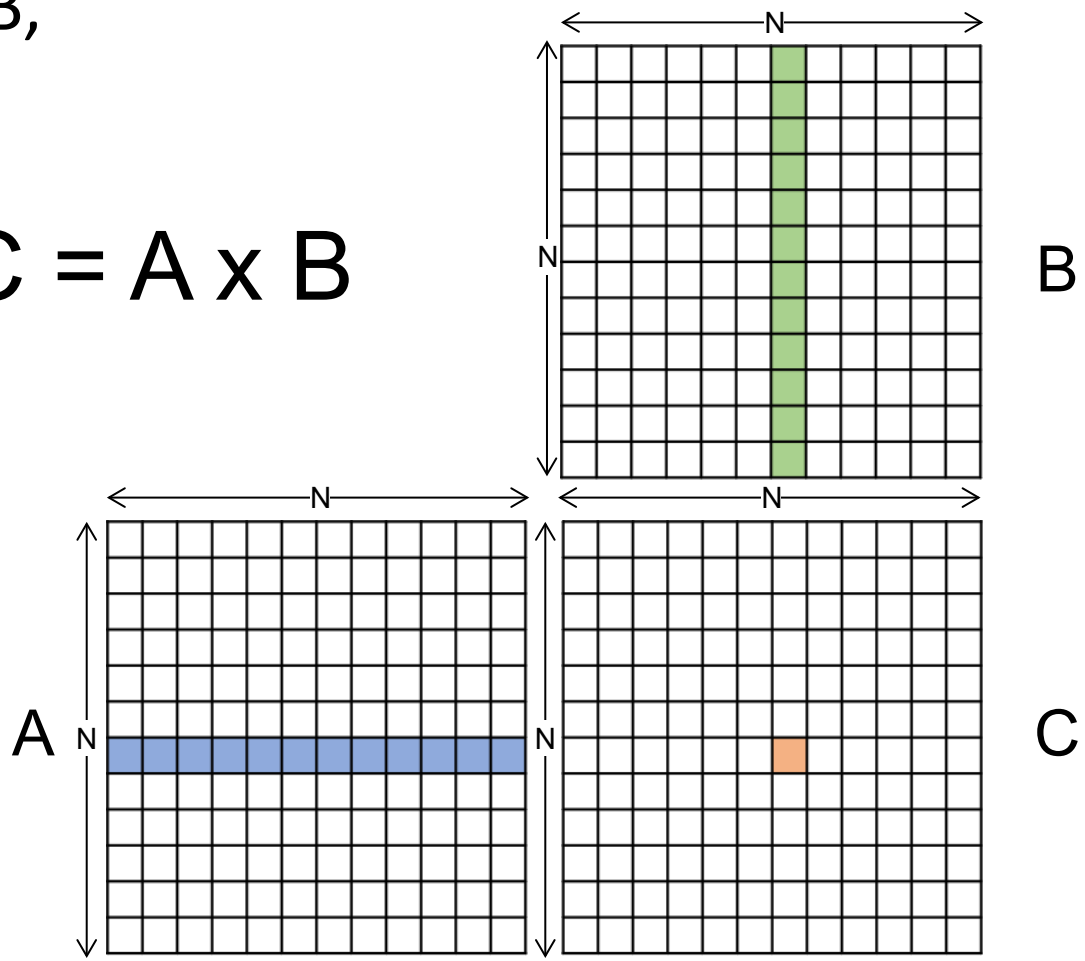
Example: Matrix-Matrix Multiplication

- Given two N × N matrices, A and B,
  - we can multiply A by B
  - to compute a third N × N matrix, P: P = AB

- In terms of the elements of P, matrix multiplication implies computing…

$$P_{ij} = \sum_{k=1}^{Width} A_{ik} B_{kj}$$

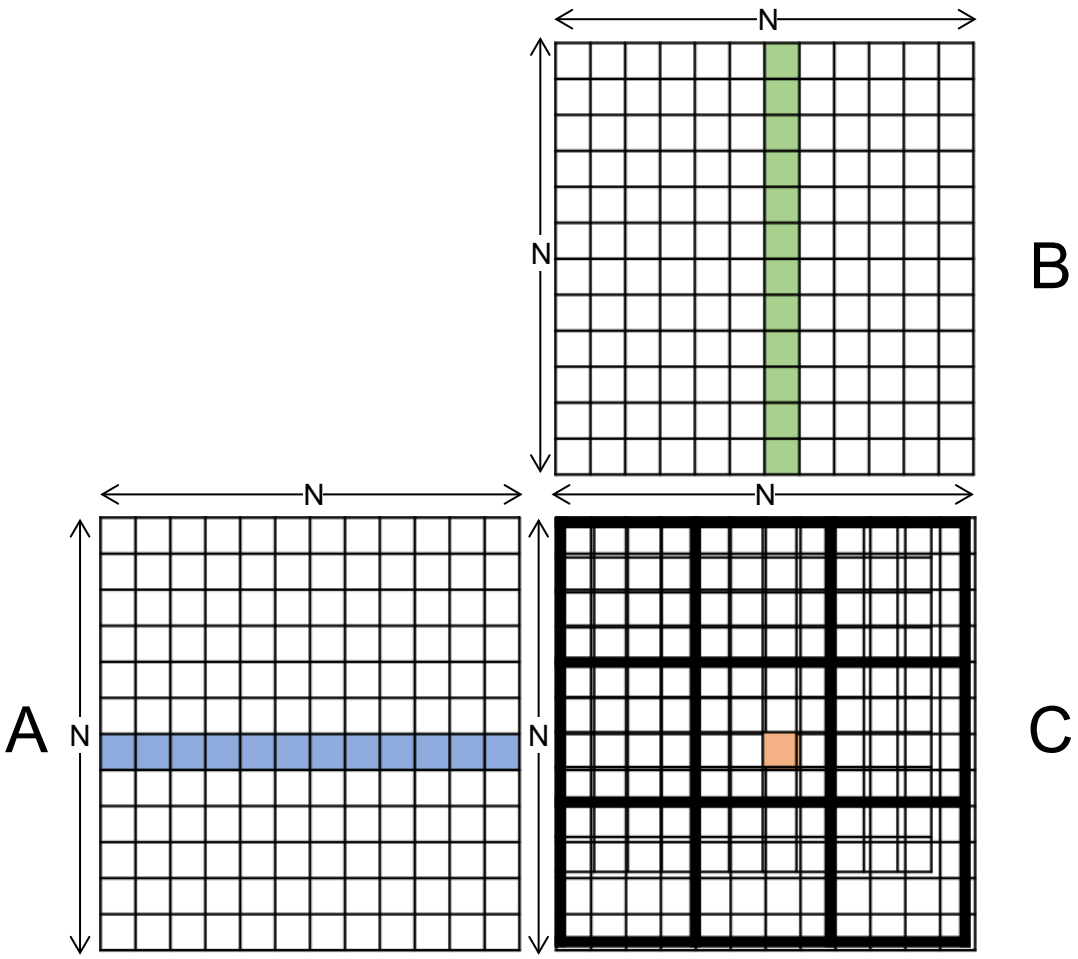
C = A x B



Parallelization Strategy

- Graphically, imagine
  - taking each element in a row of A,
  - multiplying it by the corresponding element in a column of B, and
  - summing up the products.
- Do that for every row and every column to produce P.

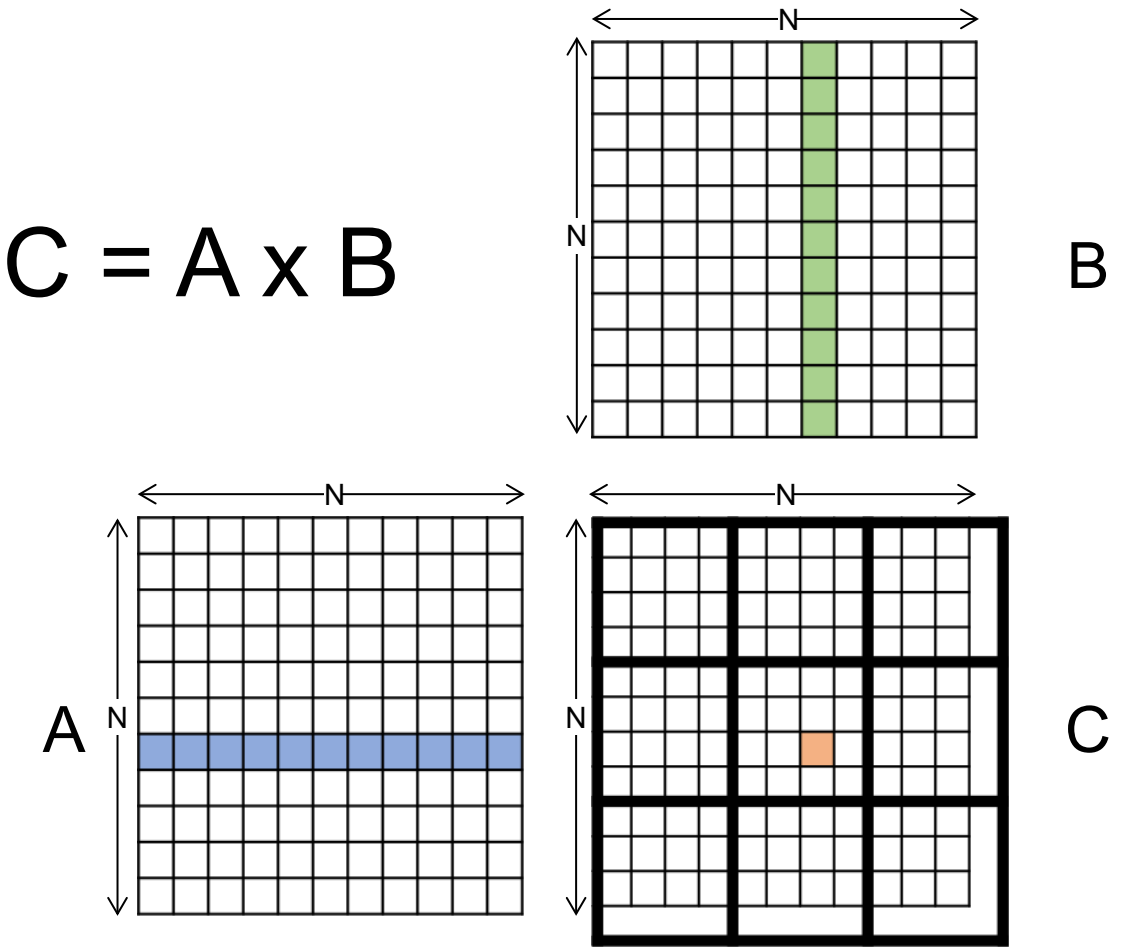
- Parallelization approach:
  - assign one **thread** to **each element** in the output matrix (C)



Simple Matrix-Matrix Multiplication Kernel Implementation

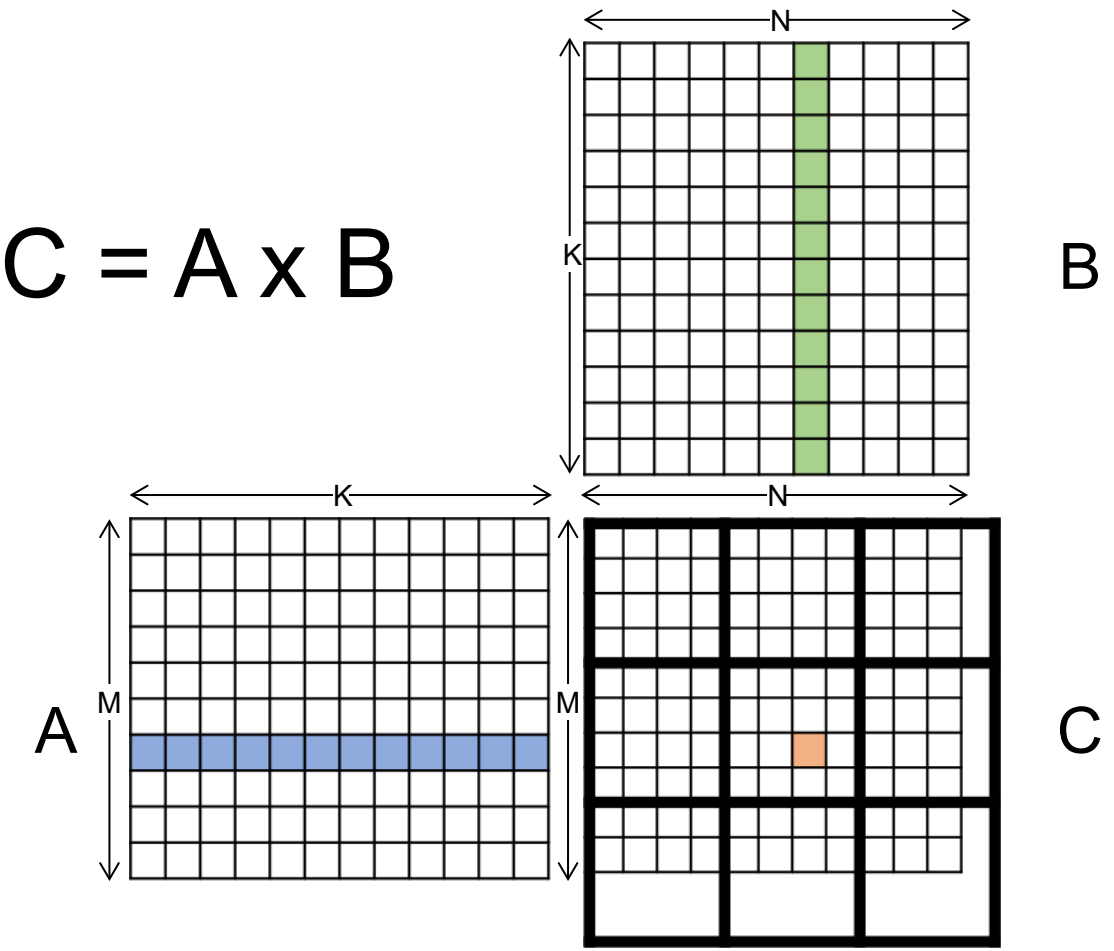
```
__global__ void mm_kernel(float* A, float* B, float* C, unsigned int N)
{
    unsigned int row = blockIdx.y*blockDim.y + threadIdx.y;
    unsigned int col = blockIdx.x*blockDim.x + threadIdx.x;

    float sum = 0.0f;

    for(unsigned int i = 0; i < N; ++i) {
        sum += A[row*N + i] * B[i*N + col];
    }

    C[row*N + col] = sum;

}
```

Issues to Consider: Boundary Conditions



Issues to Consider: Different Matrix Dimensions

$$C = A \times B$$



Issues to Consider: Performance

- Subject of a later lecture

Any More Questions?

Things to Read:

Textbook chapter 3

CUDA PG: Programming Model

Things to Do:

Get Delta account

Setup GitHub

Submit Labs 0 and 1

Problem Solving

- Q: A CUDA kernel is launched with 32 thread blocks each of which has 32×32 threads. How many threads are in a block and how many threads are launched altogether?
- A:
  - How many threads in a block will be created? 32 * 32 = 1024
  - How many threads across all blocks? 1024 * 32 = 32,768

Problem Solving

- Q: Suppose you have a 2D grid of blocks where each block is configured as a 16×16 thread block. If the grid dimensions are 32×16 blocks, calculate:
  - The total number of threads per block.
  - The total number of threads in the entire grid.
- A:
  - How many threads in a block will be created? 16 * 16 = 256
  - How many thread blocks are created? 32 * 16 = 512
  - How many threads across all blocks? 256 * 512 = 131,072

Problem Solving

- Q: You have a 3D grid of blocks with dimensions 8×4×2, and each block is configured as a 4×8×8 thread block. Calculate:
  - The total number of threads per block.
  - The total number of threads in the entire grid.
- A:
  - Total number of threads per block: 4 * 8 * 8 = 256
  - Total number of blocks in the grid: 8 * 4 * 2 = 64
  - Total number of threads in the entire grid: 256 * 64 = 16,384